Informe · Taller HTTP y MQTT

IoT 1011 · Wokwi, Render y Streamlit

Estudiante: Scandia · Noviembre 2026

Estudiante: Scandia · **Experimento:** `lab2026` · **Servidor:** `iot-backen-1011.onrender.com`
Herramientas: Wokwi (ESP32 + DHT22 simulado), backend en Render, Streamlit (Python 3.13)

Nodos propios: `N01_scandia` (exterior) · `N02_scandia` (interior) · `N03_scandia` (ventilación, opcional)

Ejercicio 1 · Comparar HTTP y MQTT

Objetivo: enviar el mismo JSON 30 veces por HTTP y 30 veces por MQTT (sobre WebSocket seguro) y comparar recepción y tamaño estimado de los mensajes.

Configuración: nodo `N01_scandia`, ronda `R01`, 30 envíos cada 2500 ms en ambos sketches. Métricas: enlace Render

Resultados

Métrica (calculada en Render)	HTTP	MQTT
Intentos en Wokwi	30	30
Intentos exitosos (`codigo=201` / `PUBLISH=SI`)	17	30
Mediciones únicas recibidas	**17**	**30**
Bytes de JSON	2240 B (≈ 132 B/medición)	3951 B (≈ 132 B/medición)
Tamaño estimado de mensajes	11083 B (≈ 652 B/medición)	5301 B (≈ 177 B/medición)
Sobrecarga estimada	8843 B (≈ 520 B/medición)	1350 B (45 B/medición)
Eficiencia estimada (JSON / mensaje)	**20,2 %**	**74,5 %**
Ventana de recepción	795,7 s	76,2 s

En HTTP, los 13 intentos fallidos aparecieron en el Serial Monitor como `codigo=-1` (*connection refused*) y `codigo=-5` (*connection lost*).

Incidencias del montaje: el primer intento con `NODO_ID = "N01_Scandia"` (con mayúscula) fue rechazado por el servidor con `422 Validation Error`. Con el ID en minúsculas (`N01_scandia`) se aceptó, lo que se verificó en `/docs` con un POST de prueba que respondió `201`. Para el sketch MQTT hubo que añadir la librería `WebSockets` en `libraries.txt`.

Evidencia: `evidencias/ej1_http_metricas.png`, `evidencias/ej1_mqtt_metricas.png`

Respuestas

¿Llegaron las 30 mediciones por ambos protocolos? No. Por HTTP llegaron 17 de 30 (57 %). Los 13 restantes fallaron antes de obtener respuesta del servidor (`connection refused` o `connection lost`), porque cada envío HTTP abre una conexión TCP + TLS nueva hacia Render a través de la red simulada de Wokwi, y cada una puede fallar de forma independiente. Por MQTT llegaron las 30 de 30: la conexión WebSocket/MQTT se abrió una sola vez y se mantuvo abierta para todas las publicaciones. La ventana mucho más larga de HTTP (795,7 s frente a 76,2 s) se explica sobre todo por los intentos fallidos que esperaban hasta el *timeout*. No debe leerse como una latencia comparable, porque los dos protocolos usan transportes distintos.

¿Cuál generó menos bytes de mensaje estimados y por qué? MQTT: 177 B por mensaje frente a 652 B en HTTP (sobrecarga de 45 B frente a ≈ 520 B; eficiencia de 74,5 % frente a 20,2 %). El JSON es el mismo en ambos casos. Cada solicitud HTTP agrega una línea de petición (`POST /mediciones HTTP/1.1`) y varias cabeceras de texto (Host, Content-Type, Content-Length, User-Agent, etc.), que suman cientos de bytes por mensaje. Un PUBLISH de MQTT solo agrega elementos binarios mínimos: - 1 byte de cabecera fija; - 2 bytes de longitud restante; - 2 bytes de longitud del tópico; - el tópico `doctorado/lab2026/N01_scandia/telemetria`, de 40 caracteres.

En total, 1 + 2 + 2 + 40 = **45 B**, exactamente la sobrecarga por mensaje que calculó Render.

¿Qué parte de esta medición corresponde realmente al JSON? Solo la fila "Bytes de JSON": unos 132 B por medición (131 B mientras la secuencia tiene un dígito y 132 B desde la secuencia 10), igual en ambos protocolos. La diferencia entre "Tamaño estimado de mensajes" y "Bytes de JSON" es la sobrecarga propia de cada protocolo: las cabeceras HTTP en un caso, la cabecera y el tópico MQTT en el otro.

¿Un `PUBLISH=SI` equivale a una medición recibida? No. `PUBLISH=SI` solo indica que el ESP32 logró escribir el paquete en su conexión WebSocket. Se publica con QoS 0, sin acuse de recibo (PUBACK), así que el dispositivo no recibe confirmación del broker. La medición solo cuenta como recibida cuando Render la registra ("Mediciones únicas recibidas"). En HTTP, en cambio, `codigo=201` sí es una respuesta del servidor que confirma que guardó el dato.

¿Por qué no es posible concluir cuál protocolo consume menos batería? Porque el backend solo estima el tamaño de la solicitud HTTP y del paquete PUBLISH. No mide el tráfico Wi-Fi ni TCP/IP, el *handshake* TLS, el encapsulado WebSocket, los *pings* de MQTT, el tiempo que la radio permanece encendida ni el consumo eléctrico real. Además, HTTP abre y cierra una conexión TLS en cada envío, mientras MQTT mantiene una conexión abierta con *keep-alive*. El consumo depende de esos factores y de la frecuencia de envío, no solo de los bytes del mensaje. Por último, en Wokwi el hardware es simulado.

Ejercicio 2 · Red MQTT compartida

Objetivo: que varios ESP32 publiquen datos distintos en tópicos propios y reciban los de los demás mediante un filtro común.

Configuración: mi nodo `N01_scandia` (perfil exterior, `INTERVALO = 5000`, `NODO_A_ESCUCHAR = "+"`), el nodo de una compañera y otros nodos del curso conectados a la misma red `lab2026` (`N01_camila`, `N03_labo`, entre otros).

- Tópico de publicación: `doctorado/lab2026/N01_scandia/telemetria`
- Filtro de suscripción: `doctorado/lab2026/+/telemetria`

Observaciones

Observación	Evidencia
Mi nodo publicó de forma continua	Monitor de N01_scandia: `MQTT n | PUBLISH=SI | JSON=149–150 B`
Mi nodo recibió a otro nodo de la red	`RECIBIDO 49–52 DE N01_camila`, con T de 25,4 a 26,5 °C
La compañera recibió mis mensajes	`RECIBIDO 248/254/259 DE N01_scandia`, con T 21,7 / 22,1 / 22,7 °C
Distintas frecuencias conviven en la red	Entre los mensajes 248 y 259, la compañera recibió 3 de N01_scandia (cada 5 s) y 9 de N03_labo
Perfiles distintos por zona	N01_scandia (exterior): T 21,7–22,7 °C, H 62–64 % · N03_labo (ventilación): T 13,6–19,6 °C, H 59–74 %

Evidencia: `evidencias/ej2_publish_recibido_N01.png`, `evidencias/ej2_publish_recibido_companera.png`

Respuestas

1. ¿Por qué `+` permite escuchar varios emisores? En MQTT los tópicos se organizan en niveles separados por `/`, y `+` es un comodín que reemplaza exactamente un nivel. Cada nodo publica en `doctorado/lab2026/<NODO_ID>/telemetria`, y el filtro `doctorado/lab2026/+/telemetria` coincide con cualquier valor en la posición del ID. Así, una sola suscripción recibe la telemetría de todos los nodos del experimento, incluso de nodos que se conecten después, sin conocerlos de antemano. Eso permitió que mi nodo recibiera a `N01_camila` sin configurar nada especial. Si se reemplaza `+` por un ID concreto (por ejemplo `N02_scandia`), el filtro coincide con un solo tópico y solo llegan los mensajes de ese nodo.

2. ¿Qué diferencia hay entre publicar y suscribirse? Publicar (PUBLISH) es enviar un mensaje al broker indicando un tópico; quien publica no sabe ni necesita saber quién lo leerá. Suscribirse (SUBSCRIBE) es registrar un filtro en el broker para que este reenvíe al cliente todos los mensajes cuyo tópico coincida. El broker es el intermediario que desacopla a emisores y receptores. Cada nodo del laboratorio cumple ambos roles: publica en su propio tópico y está suscrito al filtro común. El código descarta los mensajes cuyo `nodo_id` es el propio, y por eso cada nodo solo muestra `RECIBIDO … DE` otros nodos.

3. ¿Qué cambió al utilizar dos intervalos distintos? `INTERVALO` define cada cuánto publica cada nodo. Con 2000 ms un nodo publica unas 30 veces por minuto, y con 7000 ms unas 8 o 9 veces. El nodo rápido aparece mucho más seguido en el monitor de los demás y sus cambios (por ejemplo, subir la temperatura del DHT22) se notan antes y con más detalle, a costa de más mensajes y más tráfico. El broker no sincroniza a los nodos: cada uno publica a su propio ritmo y los receptores ven un flujo intercalado. Esto se observó en la red compartida, donde N03_labo aparecía unas tres veces más que N01_scandia.

4. ¿Al reconectar llegaron los mensajes enviados durante la desconexión o solo los nuevos? Solo los nuevos, por tres razones: - Los nodos publican con QoS 0 ("como máximo una vez"). - No usan mensajes retenidos. - Inician una sesión nueva en cada conexión, de modo que el broker no guarda mensajes para un suscriptor desconectado.

Además, un nodo detenido no mide ni publica, así que durante la pausa no se generan datos suyos. En la página `/nodos`, su última recepción queda atrasada hasta que vuelve a publicar. Al reiniciar, la secuencia comienza de nuevo y solo aparecen mediciones posteriores a la reconexión. Para no perder mensajes se necesitaría QoS 1 o 2 con sesión persistente, o mensajes retenidos (que guardan solo el último valor).

5. ¿Por qué el nodo exterior y el interior no muestran exactamente lo mismo? Aunque todos usan el mismo DHT22 simulado (24 °C y 60 % por defecto), cada sketch aplica un perfil sintético

distinto sobre la lectura: - **Exterior:** oscilación de ±2,5 °C (`2.5·sin(0.30·sec)`), y resta humedad en la misma proporción. - **Interior:** suma +2 °C con una oscilación más lenta (`2 + 1.2·sin(0.16·(sec−5))`) y tiene unos 6 puntos menos de humedad. - **Ventilación:** alterna bloques de −3 °C / +8 % de humedad.

Además, cada nodo publica en instantes distintos y con su propio contador de secuencia, así que las lecturas nunca son simultáneas. Son datos sintéticos que imitan zonas diferentes, no mediciones reales calibradas.

Síntesis (4–6 líneas): El comodín `+` reemplaza un nivel del tópico, así que `doctorado/lab2026/+/telemetria` coincide con la telemetría de cualquier nodo y una sola suscripción escucha a todos los emisores. Publicar es enviar un mensaje al broker en un tópico; suscribirse es pedirle al broker que reenvíe lo que coincida con un filtro, sin que emisor y receptor se conozcan. Con intervalos de 2000 y 7000 ms, el primer nodo publica unas 30 veces por minuto y el segundo unas 8 o 9, por lo que sus cambios se ven antes y con más detalle. Al reconectar solo llegan mensajes nuevos, porque con QoS 0, sin retención ni sesión persistente, el broker no guarda lo publicado durante la desconexión. Exterior e interior difieren porque cada sketch aplica un perfil sintético distinto sobre el DHT22 simulado y publica en instantes distintos.

Ejercicio 3 · Monitoreo y comparación con Streamlit

Objetivo: visualizar las series de cada nodo, ver el efecto del intervalo y del suavizado, comparar nodos, detectar desconexiones y exportar los datos.

Flujo de datos: 1. Los nodos publican por MQTT. 2. Render guarda cada medición con su hora de recepción (`recibido_utc`). 3. `app.py` consulta cada 3 s `GET /mediciones?experimento=lab2026&protocolo=mqtt&limite=1000`. 4. pandas ordena los datos y los convierte a hora de Chile. 5. Altair dibuja una línea por nodo.

Ejecución: con Python 3.15 no se pudo instalar `pandas<3`, porque no había versión precompilada y pedía Visual Studio para compilarla. Se instaló Python 3.13 en paralelo (`py install 3.13`) y la app se ejecutó con `py -3.13 -m streamlit run app.py`. Los pasos 3 y 4 se hicieron en la primera sesión (10-11-2026), con nodos de otros compañeros en la red. Los pasos 5 a 7 se hicieron en la segunda sesión (11-11-2026), con mis nodos `N01_scandia` (exterior, 2000 ms) y `N02_scandia` (interior, 5000 ms).

Pasos 3 y 4 · Curvas, intervalo y suavizado

Paso	Configuración	Observación
3 · Temperatura	20 min · suavizado 1	Estaciones identificadas como `nodo · zona` (N01_camila, N01_oskar, N01_scandia, N02_oskar, N03_labo). Al subir el DHT22 de N01_scandia a 34,6 °C, su línea pasó de ~25 °C a un pico de ~41 °C y luego osciló entre ~32 y 37 °C (34,6 ± 2,5 del perfil exterior).
3 · Humedad	20 min · suavizado 1	N01_scandia oscilaba entre ~56 y 64 % (60 ± 4) y, al subir la humedad del sensor, saltó a ~78 %. En el perfil exterior la humedad baja cuando sube la temperatura, y viceversa.
3 · Dos nodos propios	N01 + N02	N01_scandia (exterior) hace ondas rápidas entre ~21,5 y 26,5 °C con puntos muy juntos. N02_scandia (interior) sube lento y suave entre ~25,3 y 27,2 °C, con puntos más separados.
3 · Líneas planas	—	Al detener y reiniciar Wokwi aparecen tramos rectos largos sin puntos intermedios. Unen la última lectura antes de la pausa con la primera después: no son mediciones.
4 · Intervalo	N01: 5000 → 2000 ms	Pasó de ~12 a ~30 puntos por minuto y las ondas se ven "comprimidas" en el tiempo. Una onda del perfil son unas 21 lecturas: ~105 s con 5000 ms y ~42 s con 2000 ms, porque la oscilación depende de la secuencia y no del reloj.
4 · Suavizado	1 → 5	El pico de ~41 °C bajó a ~37,5 °C y el salto apareció por escalones. El "peine" de N03_labo se redujo y las ondas de N01_scandia se redondearon.

Paso 5 · Comparador (A = N01_scandia, B = N02_scandia, temperatura)

Momento	A − B	Desfase	Estado de los nodos	Lectura
1	−2,7 °C	0,1 s	ambos *Sin datos recientes* (~107 s)	Desfase mínimo, pero las dos lecturas tenían casi 2 minutos de antigüedad: Wokwi había pausado ambas simulaciones.
2	**−0,7 °C**	**1,5 s**	ambos *Reciente*	Comparación válida: lecturas casi simultáneas y actuales.
3	−3,0 °C	**41,5 s**	N01 *Sin datos* (63 s), N02 *Reciente* (22 s)	Comparación engañosa: se resta una lectura de N01 de hace más de un minuto con una de N02 de hace 22 s.
4	−3,5 °C	13,1 s	N01 *Sin datos* (26,9 s, umbral 15) aunque seguía encendido, N02 *Reciente*	Desfase intermedio; la última lectura de N01 venía atrasada.

En pocos minutos la diferencia pasó de −0,7 a −3,5 °C sin que cambiara ningún sensor: depende de en qué punto de su onda está cada nodo cuando se toma su última lectura.

Paso 6 · Desconexión

Umbral	Prueba	Resultado
30 s	Detener N02 y cronometrar	Pasó a *Sin datos recientes* a los **31 s**: el umbral más hasta 3 s de espera a la siguiente consulta de la app.
15 s	Detener N02 y cronometrar	Pasó a *Sin datos recientes* a los **18 s**, casi el doble de rápido que con 30 s.
15 s	Observar la tabla con los nodos encendidos	**Falsa alarma:** N01_scandia, que seguía corriendo en Wokwi, apareció como *Sin datos recientes* con 26,9 s sin datos; con umbral 30 seguiría *Reciente*. N02, que publica cada 5 s, marcaba 13,8 s, muy cerca del límite.

Incluso con los nodos encendidos, "Seg. sin datos" llegó a 8,8 s en N01 (que publica cada 2 s) y a más de 13 s en N02. La latencia de Wokwi y Render y las pausas del simulador hacen que un umbral de 15 s esté demasiado cerca del intervalo real entre recepciones.

Paso 7 · Exportación CSV (`evidencias/ej3_lab2026_lecturas.csv`)

Indicador	N01_scandia (exterior)	N02_scandia (interior)
Filas	146	31
Intervalo típico entre recepciones (mediana)	2,07 s (configurado 2000 ms)	5,17 s (configurado 5000 ms)
Temperatura (mín.–máx.)	21,51 – 26,50 °C	25,28 – 27,20 °C
Humedad (mín.–máx.)	56,0 – 64,0 %	52,0 – 55,2 %
Pausas > 30 s	9 (máx. 297 s)	7 (máx. 218 s)
Reinicios (secuencia vuelve a 1)	1	2
Saltos en `secuencia` dentro de una misma ejecución	0	0

En total son 177 mediciones más la fila de encabezados (178 filas en Excel), en la ventana de 14:44 a 15:03 UTC (11:44 a 12:03 en Chile). El CSV contiene `recibido_utc`, `nodo_id`, `zona`, `temperatura_c`, `humedad_pct` y `secuencia` de los dos nodos.

Hallazgos del análisis del CSV: - **Las pausas no fueron pérdidas de mensajes.** Aunque hubo pausas de hasta 5 minutos, la `secuencia` nunca salta (…22, 23, 24…). Durante esos tramos el simulador estaba detenido o pausado y no se generaron mediciones. Si se hubieran perdido mensajes en la red, la secuencia saltaría. - **Llegadas en ráfaga.** Las secuencias 23 a 26 de N01 se recibieron en 70 ms (14:45:28.468 a 14:45:28.538), cuando deberían estar separadas por 2 s. Quedaron retenidas en la conexión y llegaron juntas. `recibido_utc` es la hora de llegada a Render, no la hora de la medición. - **Datos sintéticos y deterministas.** Con el DHT22 fijo en 24 °C y 60 %, los valores dependen solo de la secuencia: tras cada reinicio N02 repite exactamente 25,28337 °C y 55,19439 % en la secuencia 1, y los extremos de N01 son exactamente 24 ± 2,5 °C y 60 ± 4 %.

Evidencia: `ej3_curvas_temperatura.png`, `ej3_curvas_humedad.png`, `ej3_intervalo2000_suavizado1.png`, `ej3_suavizado5.png`, `ej3_curvas_dos_nodos.png`, `ej3_comparador.png`, `ej3_comparador_reciente.png`, `ej3_estado.png`, `ej3_estado_umbral15.png`, `ej3_lab2026_lecturas.csv`

Respuestas

1. ¿Qué representan los puntos y las líneas? Cada punto es una medición real que llegó a Render desde un nodo, ubicada en el eje X según su hora de recepción (en hora de Chile) y en el eje Y según su valor. La línea solo une puntos consecutivos del mismo nodo para mostrar la tendencia; lo que hay entre dos puntos es una interpolación visual, no una medición. En mis gráficos se ve con claridad: cada vez que Wokwi se pausó apareció un tramo recto de varios minutos sin puntos, que solo une la lectura anterior a la pausa con la siguiente. Cada color corresponde a una estación (`nodo_id · zona`), y el gráfico no alinea instantes entre nodos: cada uno conserva sus tiempos reales. Con suavizado mayor que 1, el punto ya no es la lectura cruda, sino un promedio móvil.

2. ¿Por qué importa el desfase de tiempo al comparar dos nodos? El comparador resta la última lectura de A y la de B, pero esas lecturas no son simultáneas. En mi prueba obtuve −0,7 °C con 1,5 s de desfase (válido) y −3,0 °C con 41,5 s de desfase (engañoso): en el segundo caso se comparaba una lectura de N01 de hace más de un minuto con otra de N02 mucho más reciente. Como el perfil exterior recorre 5 °C en unos 40 s, unos pocos segundos ya cambian la diferencia. Además, un desfase pequeño no garantiza que los datos sean actuales: obtuve −2,7 °C con solo 0,1 s de desfase, pero con ambas lecturas de hace casi 2 minutos. Para comparar bien hay que mirar el desfase entre los nodos y también la antigüedad de cada lectura.

3. ¿Qué efecto tiene suavizar los datos? El suavizado es un promedio móvil de las últimas *n* lecturas de cada nodo. Reduce el ruido y facilita ver la tendencia, pero tiene tres costos: - **Atenúa los picos:** en mi prueba, el máximo bajó de ~41 °C a ~37,5 °C con suavizado 5. - **Retrasa los cambios bruscos:** un salto se ve por escalones durante unas *n* lecturas. - **Puede ocultar eventos cortos.**

El efecto depende del intervalo: 5 lecturas son ~10 s en N01 (2 s) y ~26 s en N02 (5 s). Y como el promedio se hace por lecturas y no por tiempo, después de una pausa los primeros puntos nuevos se mezclan con valores de minutos atrás. El valor suavizado no es una medición nueva, solo una forma de presentarla.

4. ¿Cómo se diferencia una estación desconectada de una que transmite una temperatura constante? Por la llegada de datos, no por el valor. Una estación con temperatura constante sigue enviando mensajes: aparecen puntos nuevos (sobre una línea plana), la `secuencia` avanza y los "segundos sin datos" se mantienen bajo el umbral, por lo que su estado es *Reciente*. Una estación desconectada deja de generar puntos y la edad de su última lectura crece hasta superar el umbral. Al detener N02, pasó a *Sin datos recientes* a los 31 s con umbral 30 y a los 18 s con umbral 15. Su último valor puede verse perfectamente normal; por eso el estado se calcula con el tiempo transcurrido y no con la medición. El umbral debe ser bastante mayor que el intervalo de envío más la latencia: con 15 s la desconexión real se detectó en 18 s, pero N01 se marcó como caído (26,9 s sin datos) mientras seguía encendido, una falsa alarma, y N02 rozaba el límite con 13,8 s. Se detecta antes, pero aumentan las falsas alarmas. En el CSV, además, la `secuencia` permite distinguir una pausa del emisor (sin saltos) de mensajes perdidos (con saltos).

5. ¿Qué aporta la exportación CSV a un análisis posterior? Deja un registro de los datos crudos (hora de recepción, nodo, zona, temperatura, humedad y secuencia) en un formato que se abre en Excel, Python o R. Con mi CSV de 177 mediciones pude: - calcular el intervalo real de cada nodo (2,07 s y 5,17 s); - verificar los rangos de cada perfil; - contar las pausas y los reinicios; - comprobar con la `secuencia` que no hubo mensajes perdidos dentro de cada ejecución; - detectar mensajes que llegaron en ráfaga.

Nada de eso se ve a simple vista en el panel. El CSV permite además documentar y compartir el experimento sin depender de que Render siga disponible. Como exporta solo la ventana y los nodos seleccionados, conviene elegirlos bien antes de descargar.

Conclusiones

- Con el mismo JSON (~132 B), **MQTT fue más eficiente que HTTP** en tamaño estimado (74,5 % frente a 20,2 %) y **más confiable** en este entorno (30/30 frente a 17/30), porque mantiene una conexión abierta en lugar de abrir una por envío. Estas cifras son estimaciones de mensaje y no permiten concluir nada sobre consumo de energía.
- El modelo **publicar/suscribir con comodines** permite que nodos que no se conocen formen una red: basta con compartir una convención de tópicos y un broker.
- En el monitoreo, **el intervalo de envío, el suavizado, el desfase y el umbral de desconexión** cambian lo que se ve tanto como los datos mismos. La misma pareja de nodos mostró diferencias de −0,7 a −3,5 °C según el instante, y un umbral de 15 s generó alertas en nodos que seguían vivos. Interpretar bien un panel IoT exige saber cómo se generan y transportan las mediciones.
- El **CSV con número de secuencia** permite ir más allá del panel: separar pausas del emisor de pérdidas en la red y medir el intervalo real de cada nodo.

Anexo · Evidencias

```
Detalle HTTP: connection refused
HTTP 2/30 | codigo=201 | JSON=131 B
HTTP 3/30 | codigo=-1 | JSON=131 B
Detalle HTTP: connection refused
HTTP 4/30 | codigo=-1 | JSON=131 B
Detalle HTTP: connection refused
HTTP 5/30 | codigo=-5 | JSON=131 B
Detalle HTTP: connection lost
HTTP 6/30 | codigo=201 | JSON=131 B
HTTP 7/30 | codigo=201 | JSON=131 B
HTTP 8/30 | codigo=-5 | JSON=131 B
Detalle HTTP: connection lost
HTTP 9/30 | codigo=201 | JSON=131 B
HTTP 10/30 | codigo=-1 | JSON=132 B
Detalle HTTP: connection refused
HTTP 11/30 | codigo=-1 | JSON=132 B
Detalle HTTP: connection refused
```

Figura 1. Ej. 1 · Envíos HTTP: respuestas 201 y fallos -1 / -5 (ej1_http_envios.png)

```
rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
WiFi: conectando..... OK
HTTP: 30 mediciones. La API calculara las metricas.
HTTP 1/30 | codigo=422 | JSON=131 B
HTTP 2/30 | codigo=-1 | JSON=131 B
Detalle HTTP: connection refused
HTTP 3/30 | codigo=422 | JSON=131 B
HTTP 4/30 | codigo=422 | JSON=131 B
HTTP 5/30 | codigo=422 | JSON=131 B
HTTP 6/30 | codigo=-1 | JSON=131 B
Detalle HTTP: connection refused
HTTP 7/30 | codigo=-1 | JSON=131 B
Detalle HTTP: connection refused
```

Figura 2. Ej. 1 · Rechazo 422 con el ID en mayúscula (ej1_http_error_422.png)

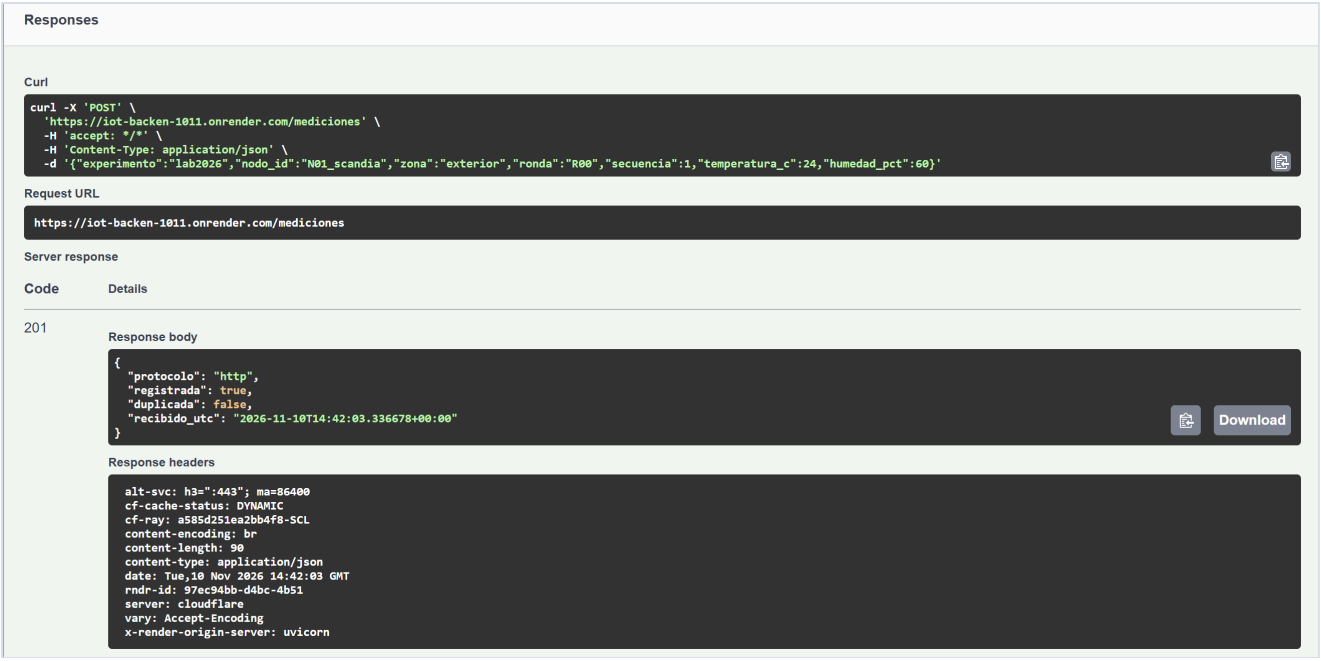


Figura 3. Ej. 1 · POST de prueba en /docs aceptado (201) (ej1_docs_post_prueba_201.png)

```
METRICAS HTTP GET: 200
METRICAS DEL BACKEND: lab2026 / N01_scandia / R01
Recibidas = mediciones UNICAS; tamano estimado NO es trafico de red.
HTTP: 17 recibidas | JSON 2240 B | mensaje estimado 11083 B | sobrecarga
estimada 8843 B | eficiencia 20.2% | ventana 795.7 s
MQTT: 0 recibidas
Excluye TCP/IP, WiFi, TLS, conexion, retransmisiones y energia.
```

Figura 4. Ej. 1 · Métricas de Render tras los 30 envíos HTTP (ej1_http_metricas.png)

```
METRICAS EN RENDER: HTTP 200
METRICAS DEL BACKEND: lab2026 / N01_scandia / R01
Recibidas = mediciones UNICAS; tamano estimado NO es trafico de red.
HTTP: 17 recibidas | JSON 2240 B | mensaje estimado 11083 B | sobrecarga
estimada 8843 B | eficiencia 20.2% | ventana 795.7 s
MQTT: 30 recibidas | JSON 3951 B | mensaje estimado 5301 B | sobrecarga estimada
1350 B | eficiencia 74.5% | ventana 76.2 s
Excluye TCP/IP, WiFi, TLS, conexion, retransmisiones y energia.
```

Figura 5. Ej. 1 · Métricas de Render: HTTP vs MQTT (ej1_mqtt_metricas.png)

```
doctorado/lab2026/N01_camila/telemetria
MQTT 12 | PUBLISH=SI | JSON=149 B
RECIBIDO 49 DE N01_camila | T=25.4 C |
H=57.7 % |
doctorado/lab2026/N01_camila/telemetria
MQTT 13 | PUBLISH=SI | JSON=150 B
RECIBIDO 50 DE N01_camila | T=26.0 C |
H=56.8 % |
doctorado/lab2026/N01_camila/telemetria
MQTT 14 | PUBLISH=SI | JSON=150 B
RECIBIDO 51 DE N01_camila | T=26.3 C |
H=56.2 % |
doctorado/lab2026/N01_camila/telemetria
MQTT 15 | PUBLISH=SI | JSON=150 B
RECIBIDO 52 DE N01_camila | T=26.5 C |
H=56.0 % |
doctorado/lab2026/N01_camila/telemetria
MQTT 16 | PUBLISH=SI | JSON=150 B
```

Figura 6. Ej. 2 · Monitor de N01_scandia: PUBLISH=SI y RECIBIDO DE N01_camila (ej2_publish_recibido_N01.png)

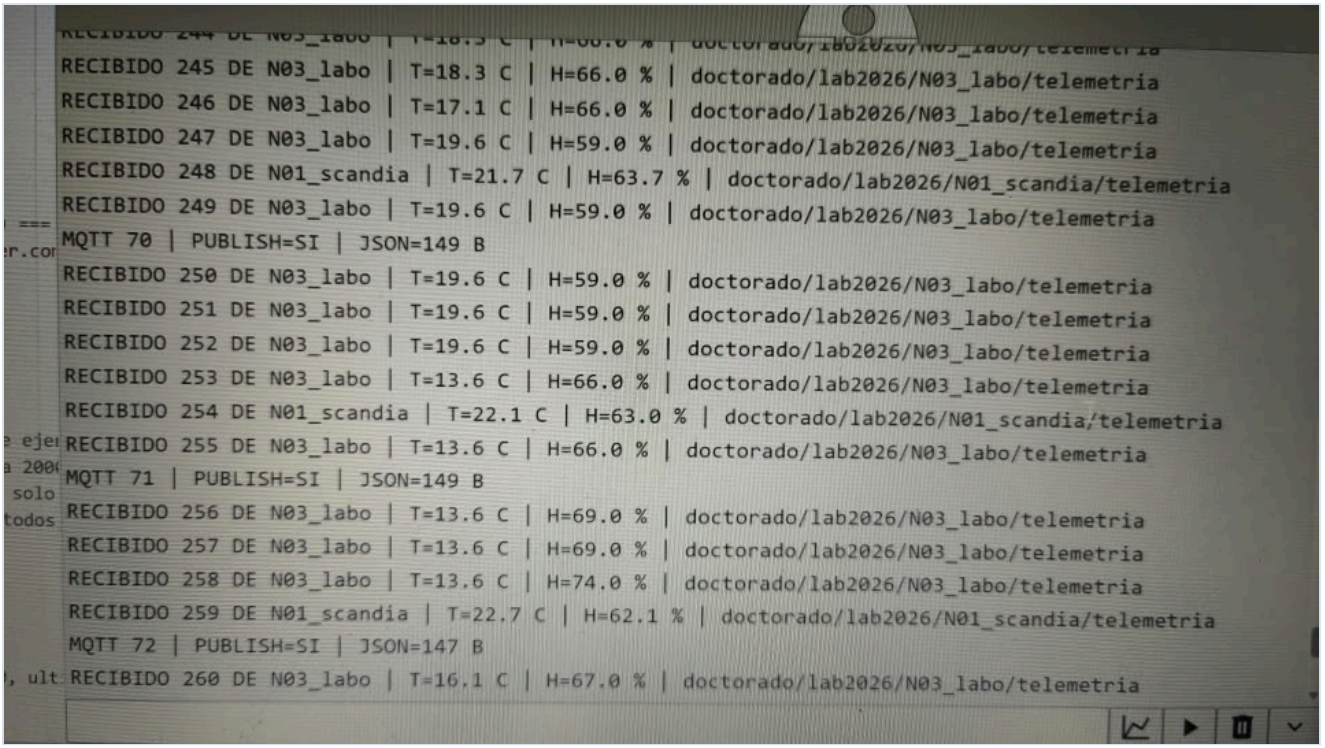


Figura 7. Ej. 2 · Monitor de la compañera: RECIBIDO DE N01_scandia y N03_labo (ej2_publish_recibido_companera.png)

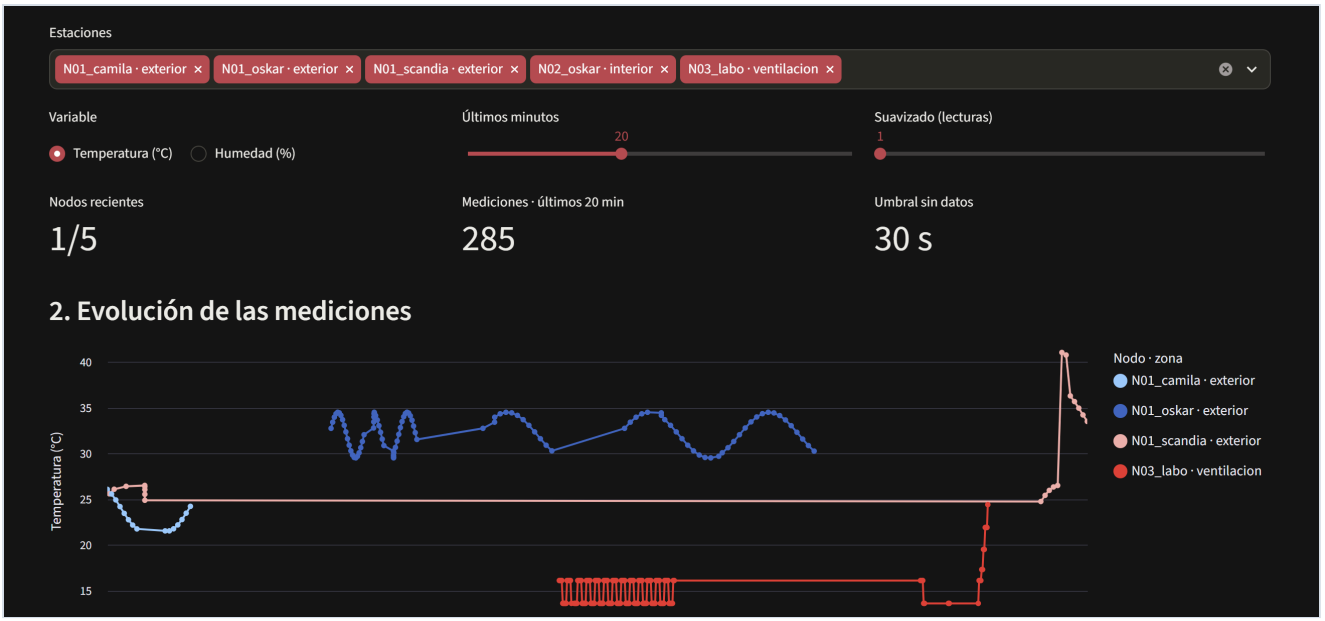


Figura 8. Ej. 3 · Curvas de temperatura por estación (DHT22 subido a 34,6 °C) (ej3_curvas_temperatura.png)

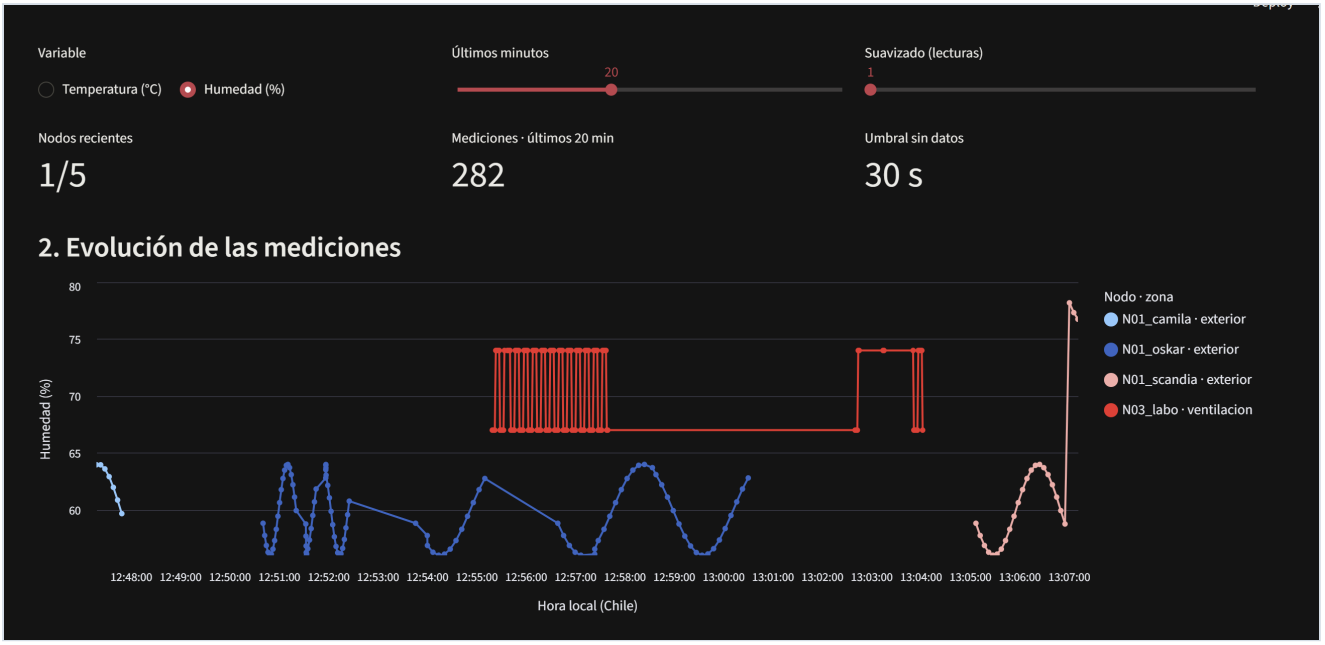


Figura 9. Ej. 3 · Curvas de humedad por estación (ej3_curvas_humedad.png)

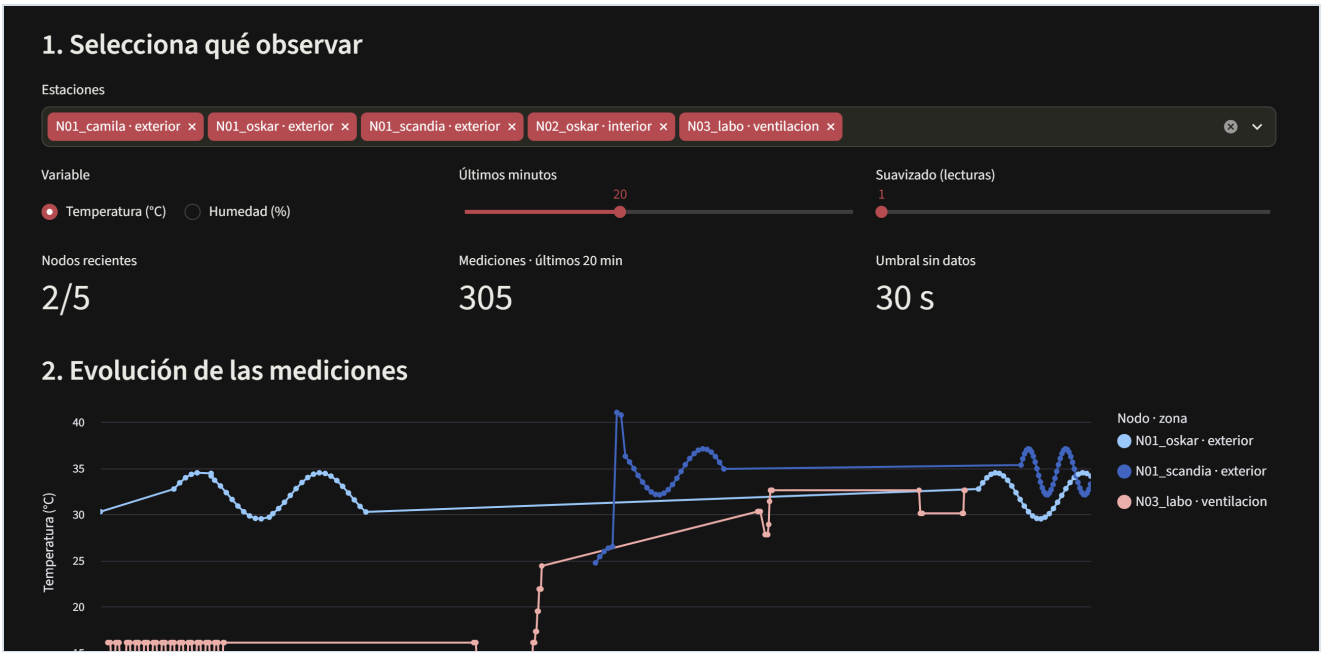


Figura 10. Ej. 3 · N01 con INTERVALO 2000 ms, suavizado 1 (ej3_intervalo2000_suavizado1.png)

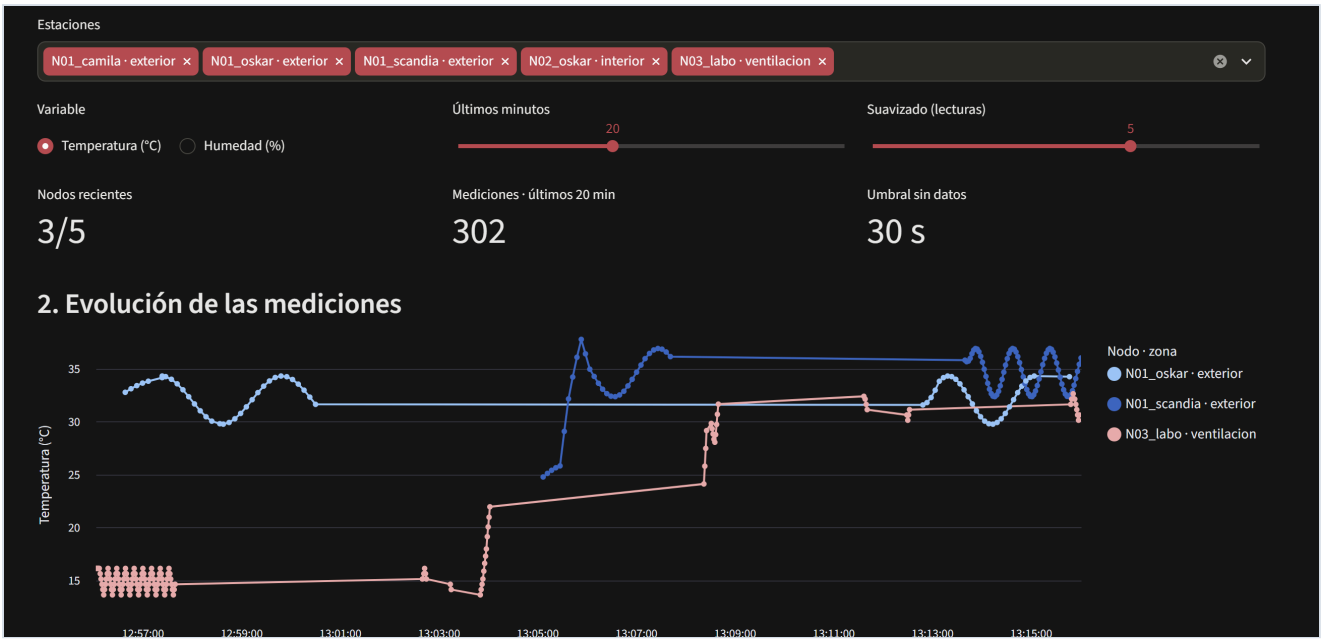


Figura 11. Ej. 3 · Suavizado 5 (ej3_suavizado5.png)

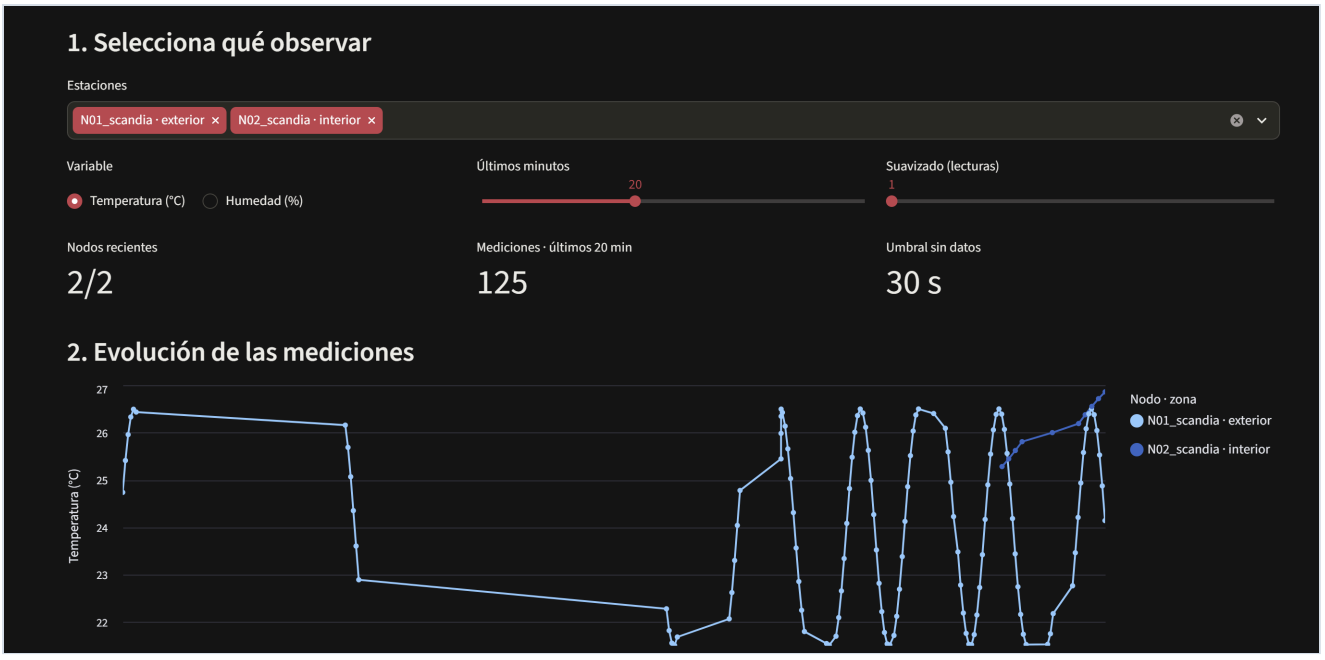


Figura 12. Ej. 3 · N01_scandia (exterior) y N02_scandia (interior) (ej3_curvas_dos_nodos.png)



Figura 13. Ej. 3 · Comparador: −2,7 °C, desfase 0,1 s, ambos sin datos recientes (ej3_comparador.png)

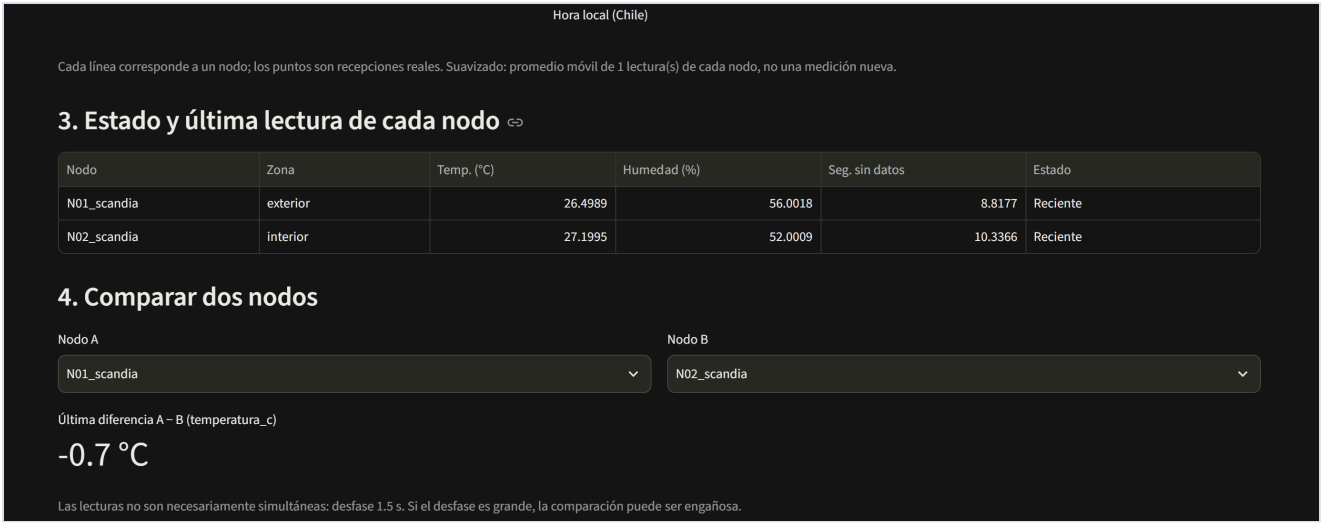


Figura 14. Ej. 3 · Comparador: −0,7 °C, desfase 1,5 s, ambos recientes (ej3_comparador_reciente.png)



Figura 15. Ej. 3 · Umbral 30 s: N01 sin datos, desfase 41,5 s (ej3_estado.png)



Figura 16. Ej. 3 · Umbral 15 s: falsa alarma en N01 (ej3_estado_umbral15.png)